Lumina Chronica

Master Project Bible

Teil 4 – Database Design & API Specification

---

43. Database Philosophy

43.1 Grundprinzip

Lumina Chronica trennt strikt zwischen:

1. Inhaltlichen Daten
2. Benutzerbezogenen Daten
3. Dateien
4. Berechtigungen

Beispiel:

Ein Buch besteht aus:

```
Buchinhalt
+
Metadaten
+
Benutzerinteraktionen
+
Datei
```

Diese Informationen werden getrennt gespeichert.

43.2 Storage Separation

Cloudflare D1

Verwendet für:

- Benutzer
- Bücherinformationen
- Beziehungen

- Fortschritte
- Einstellungen
- Projekte

Cloudflare R2

Verwendet für:

- EPUB Dateien
- PDFs
- Cover
- Bilder
- Karten
- Projektdateien

---

44. Entity Relationship Overview

Gesamtübersicht:

```
USER

 |
 |
 +---- BOOK
 |
 |
 +---- SHELF
 |
 |
 +---- PROJECT
 |
 |
 +---- READING_PROGRESS
 |
 |
 +---- STATISTICS


BOOK

 |
 |
 +---- BOOK_FILE
 |
 |
 +---- BOOK_TAG
```

```
 |
 |
 +---- BOOK_RATING
 |
 |
 +---- COMMENT


PROJECT

 |
 |
 +---- CHARACTER
 |
 |
 +---- LOCATION
 |
 |
 +---- TIMELINE_EVENT
 |
 |
 +---- PROJECT_FILE
```

---

45. Database Tables

45.1 Users Table

Zweck

Speichert alle Benutzerkonten.

Tabelle

`users`

Felder

Feld	Typ	Beschreibung
id	INTEGER	Primärschlüssel
username	TEXT	Öffentlicher Name

Feld	Typ	Beschreibung
email	TEXT	Login-Adresse
password_hash	TEXT	Verschlüsseltes Passwort
avatar_url	TEXT	Profilbild
role_id	INTEGER	Benutzerrolle
created_at	DATETIME	Erstellung
last_login	DATETIME	Letzte Anmeldung

Beispiel:

```
{
"id":1,
"username":"Matthias",
"email":"example@mail.com",
"role":"USER"
}
```

45.2 Roles Table

Zweck

Berechtigungssystem.

Tabelle:

`roles`

Felder:

Feld	Typ
id	INTEGER
name	TEXT
permissions	JSON

Beispiele:

```
USER

AUTHOR

MODERATOR

ADMIN
```

45.3 User Settings Table

Zweck

Persönliche Einstellungen.

Tabelle:

`user_settings`

Felder:

Feld	Typ
id	INTEGER
user_id	INTEGER
theme	TEXT
reader_mode	TEXT
font_size	INTEGER
language	TEXT

Beispiel:

```
{
"theme":"paper",
"reader_mode":"book",
"font_size":18
}
```

---

46. Book System Database

46.1 Books Table

Zweck

Speichert grundlegende Buchinformationen.

Tabelle:

`books`

Felder:

Feld	Typ	Beschreibung
id	INTEGER	ID
owner_id	INTEGER	Besitzer
title	TEXT	Titel
author	TEXT	Autor
description	TEXT	Beschreibung
cover_url	TEXT	Cover
language	TEXT	Sprache
genre	TEXT	Genre
visibility	TEXT	Sichtbarkeit
created_at	DATETIME	Datum

Visibility:

```
PRIVATE

SHARED

PUBLIC
```

46.2 Book Files Table

Zweck

Verknüpft Buch mit gespeicherter Datei.

Tabelle:

`book_files`

Felder:

Feld	Typ
id	INTEGER
book_id	INTEGER
file_url	TEXT
format	TEXT
size	INTEGER

Beispiel:

```
{
"format":"EPUB",
"size":523400
}
```

46.3 Book Metadata Table

Zweck

Zusätzliche Informationen.

Tabelle:

`book_metadata`

Felder:

Feld	Typ
id	INTEGER
book_id	INTEGER
isbn	TEXT
publisher	TEXT
release_date	DATE
pages	INTEGER

46.4 Tags System

Tags Table

`t`
`ags`

Felder:

Feld	Typ
id	INTEGER
name	TEXT

Beispiele:

```
Fantasy

Romance

Science Fiction

History
```

Book Tags Table

Viele-zu-viele Beziehung.

`t_book_tags`

Feld
book_id
tag_id

47. Shelf System

47.1 Shelves Table

Zweck

Persönliche Buchsammlungen.

`shelves`

Felder:

Feld	Typ
id	INTEGER
owner_id	INTEGER
name	TEXT
description	TEXT
cover_url	TEXT
visibility	TEXT

Beispiel:

```
Fantasy Sammlung

Schulbücher

Lieblingswerke
```

47.2 Shelf Books

Viele-zu-viele Beziehung.

`shelf_books`

Felder:

Feld
shelf_id
book_id

---

48. Reading System

48.1 Reading Progress Table

Zweck

Speichert persönlichen Lesestand.

`reading_progress`

Felder:

Feld	Typ
id	INTEGER
user_id	INTEGER
book_id	INTEGER
chapter	INTEGER
position	FLOAT
percentage	FLOAT
last_opened	DATETIME

Beispiel:

```
{
"book":"Der Hobbit",
"chapter":5,
"percentage":67
}
```

48.2 Bookmarks Table

Spätere Version.

`bookmarks`

Felder:

Feld
id
user_id
book_id
location
note

48.3 Highlights Table

Spätere Version.

Speichert:

- markierte Textstellen
- persönliche Notizen

---

49. Project / Worldbuilding Database

49.1 Projects Table

Zweck

Grundelement jedes kreativen Projekts.

`projects`

Felder:

Feld	Typ
id	INTEGER
owner_id	INTEGER
title	TEXT
description	TEXT
type	TEXT
cover_url	TEXT
visibility	TEXT

Projektarten:

```
WORLD

NOVEL

RPG

CUSTOM
```

49.2 Project Members

Für gemeinsame Projekte.

`project_members`

Felder:

Feld
project_id
user_id
permission

Permissions:

```
VIEW

EDIT

OWNER
```

49.3 Characters Table

Zweck

Charakterverwaltung.

`characters`

Felder:

Feld
id
project_id
name
description
image_url
age
origin
personality
biography

49.4 Locations Table

Zweck

Orte einer Welt.

`locations`

Felder:

Feld
id
project_id
name
description
image_url
coordinates

49.5 Timeline Table

Zweck

Historische Ereignisse.

`timeline_events`

Felder:

Feld
id
project_id
title
description
date

49.6 Project Files

Speichert:

- Karten
- Dokumente
- Bilder

`project_files`

---

50. Community Database

50.1 Following System

User follows User

`followers`

Felder:

Feld
follower_id
following_id
created_at

50.2 Ratings

`ratings`

Felder:

Feld
id

Feld
user_id
book_id
rating
created_at

Rating:

```
1-5 Sterne
```

50.3 Comments

Spätere Version.

`comments`

Felder:

Feld
id
user_id
target_type
target_id
content
created_at

---

51. Statistics Database

User Statistics

`user_statistics`

Speichert:

- gelesene Bücher
- Lesedauer
- Seiten
- Aktivität

Beispiel:

```
{
"booksRead":50,
"readingTime":3200
}
```

---

52. API Specification

API Standard

Format:

REST + JSON

Base URL:

```
/api
```

---

53. Authentication API

Register

```
POST /api/auth/register
```

Request:

```
{
"username":"User",
"email":"mail@test.com",
"password":"password"
}
```

Login

```
POST /api/auth/login
```

Response:

```
{
"token":"jwt-token",
"userId":1
}
```

---

54. Book API

Get Books

```
GET /api/books
```

Parameter:

```
?page=1
&genre=fantasy
&search=dragon
```

Upload Book

```
POST /api/books/upload
```

Upload:

- Datei

• Metadaten

Get Single Book

```
GET /api/books/{id}
```

Delete Book

```
DELETE /api/books/{id}
```

---

55. Reader API

Get Progress

```
GET /api/reading/{bookId}
```

Update Progress

```
POST /api/reading/update
```

Body:

```
{
"bookId":5,
"chapter":3,
"percentage":42
}
```

---

56. Project API

Create Project

```
POST /api/projects
```

Get Projects

```
GET /api/projects
```

Add Character

```
POST /api/projects/{id}/characters
```

---

57. Permission Logic

Jede Anfrage prüft:

```
Ist User eingeloggt?

↓

Hat User Zugriff?

↓

Welche Rechte besitzt User?

↓

Aktion erlauben/verweigern
```

---

58. Database Rules

Wichtig

Nie:

- Dateien direkt in D1 speichern

- Passwörter unverschlüsselt speichern
- private Inhalte ohne Prüfung zurückgeben

Database Philosophy Summary

```
D1
=
Information


R2
=
Files


API
=
Security + Logic


Frontend
=
Experience
```

End of Part 4